Messa
Multi-Agent development in Python

Dr. Kaneeka Vidanage.

Imports purposes

```
from mesa import Agent, Model
from mesa.time import RandomActivation
from mesa.space import MultiGrid
from mesa.datacollection import DataCollector
import numpy as np
```

- Agents represents the customers.
- Model represents the cinema hall`s simulation environment.
- **RandomActivation** triggers random activation of agents to simulate random customers attempt to book seats.
- **MultiGrid**, represents the seats with rows and columns in the cinema hall environment. Used to model the spatial context.
- DataCollector, responsible to collecting the data generated during the simulation process.

Cinema Hall Simulation

```
class CinemaModel(Model):
    def __init__(self, N, width, height):
        self.num_agents = N
        self.grid = MultiGrid(width, height, True)
        self.schedule = RandomActivation(self)

        # Create agents
        for i in range(self.num_agents):
            a = CustomerAgent(i, self)
            x = self.random.randrange(self.grid.width)
            y = self.random.randrange(self.grid.height)
            self.grid.place_agent(a, (x, y))
            self.schedule.add(a)

        # Data collector
        self.datacollector = DataCollector(
            agent_reporters={"Booking": "booking"}
        )

    def step(self):
        self.datacollector.collect(self)
        self.schedule.step()
```

- Inherits from the Model class.
- **def __init__(self, N, width, height):** → Initializes the model. This is the constructor of the class.
- N →Number of customers / Agents.
- Width & Height → Cinema hall environment capacity.
- **self**→ Similar to ***this*** key in Java. Refers to the current object.
- **self.num_agents = N** →Stores the number of agents.
- **self.grid = MultiGrid(width, height, True):** → Arranges the cinema hall space into rows / columns with seating arrangements.
- **True** → Agents can move in between rows / columns. Same are in a cinema hall.
- **RandomActivation**→ Randomly select agents

Agent dispositioning at the cinema hall.

Agent Creation:

- `` `for i in range(self.num_agents):` ``: Loop to create `` `CustomerAgent` `` instances.
- `` `a = CustomerAgent(i, self)` ``: Creates a `` `CustomerAgent` `` with a unique ID.
- `` `x = self.random.randrange(self.grid.width)` ``: Randomly selects a column index.
- `` `y = self.random.randrange(self.grid.height)` ``: Randomly selects a row index.
- `` `self.grid.place_agent(a, (x, y))` ``: Places the agent at the random grid location.
- `` `self.schedule.add(a)` ``: Adds the agent to the scheduler.

Data collection & moment mover

Data Collector:

- `self.datacollector = DataCollector(agent_reporters={"Booking": "booking"})`: Initializes a `DataCollector` to track the `booking` attribute of each agent.

`def step(self):`: Defines the `step` method, which advances the simulation by one step.

- `self.datacollector.collect(self)`: Collects data for this step.
- `self.schedule.step()`: Advances the scheduler, activating each agent.

Progressive time stamps to see, what`s happening from moment to moment when agents randomly occupy the seats.

Customer Agent

```python
class CustomerAgent(Agent):
    """ An agent with random booking behavior. """
    def __init__(self, unique_id, model):
        super().__init__(unique_id, model)
        self.name = f"Customer_{unique_id}"
        self.booking = 0

    def step(self):
        x = self.random.randrange(self.model.grid.width)
        y = self.random.randrange(self.model.grid.height)

        # Try to book a seat
        cell_content = self.model.grid.get_cell_list_contents([(x, y)])
        if len(cell_content) == 0:
            self.booking = 1
            print(f"Customer {self.name} successfully booked seat ({x}, {y})")
        else:
            self.booking = 0
            print(f"Customer {self.name} failed to book seat ({x}, {y})")
```

`class CustomerAgent(Agent)`: Defines the `CustomerAgent` class, which extends the `Agent` class from Mesa. Each agent represents a customer trying to book a seat.

`def __init__(self, unique_id, model):`: Initializes the agent with a unique ID and the model reference.

- `self.name = f"Customer_{unique_id}"`: Sets a name for the agent.
- `self.booking = 0`: Initializes the booking status (0 means not booked).

- Represents, an agent choosing of a seat randomly.
- Seat row and column corresponds with X & Y values.
- Check if the randomly selected seat by specifying X & Y are booked or not.

Booking Attempt.

Booking Attempt:

- `if len(cell_content) == 0:`: Checks if the cell is empty (i.e., no agent is occupying it).
- `self.booking = 1`: Marks the seat as booked.
- `print(f"Customer {self.name} successfully booked seat ({x}, {y})")`: Prints a success message.
- `else:`: If the cell is occupied, the booking fails.
- `self.booking = 0`: Marks the booking as failed.
- `print(f"Customer {self.name} failed to book seat ({x}, {y})")`: Prints a failure message.

Simulation execution

`width = 10` and `height = 10`: Define the size of the grid.

`num_agents = 15`: Define the number of customer agents.

`model = CinemaModel(num_agents, width, height)`: Creates an instance of the `CinemaModel`.

`for i in range(20): model.step()`: Runs the simulation for 20 steps.

`for agent in model.schedule.agents:`: Iterates over all agents to print their final booking status.

```
# Create the model and run the simulation
width = 10
height = 10
num_agents = 15
model = CinemaModel(num_agents, width, height)

for i in range(20):
    model.step()

# Print final booking results
for agent in model.schedule.agents:
    print(f"Customer {agent.unique_id} booking status: {agent.booking}")
```

Unable to book ?

- Given a 10x10 grid with seats ranging from (0,0) to (9,9), there are a total of 10×10=10010 \times 10 = 10010×10=100 seats available.
- With 15 agents, it might seem like all agents should be able to find a seat.
- But the situation can be more complex due to the random nature of the booking attempts and the way the model is set up.
- Assuming each agent randomly selects a seat and there are multiple agents making booking attempts simultaneously.
  - **Agent 1** chooses seat (3, 5) and books it.
  - **Agent 2** chooses seat (3, 5) and fails because it's already booked.
  - **Agent 3** chooses seat (7, 2) and books it.
  - **Agent 4** chooses seat (7, 2) and fails because it's already booked
- Chosen cells are currently occupied or already booked by other agents in previous steps

Multiple Steps with More Opportunities ??

```
for i in range(1):
    model.step()
```

- When you increase the range from 1 to 20 in the loop that calls model.step(), you are running the simulation for more steps.
- **Single Step (range(1)):**
  - In a single step, each agent makes a booking attempt, and if multiple agents choose the same seat, only one succeeds. Consequently, some agents might fail to book seats due to conflicts.
- **Multiple Steps (range(20)):**
  - Running the simulation for 20 steps allows agents to attempt booking multiple times. This means that if an agent fails to book a seat in one step due to competition, it might have another opportunity in the next step.